

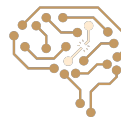
Large Language Model (LLM)

Arash Marioriyad

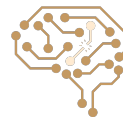
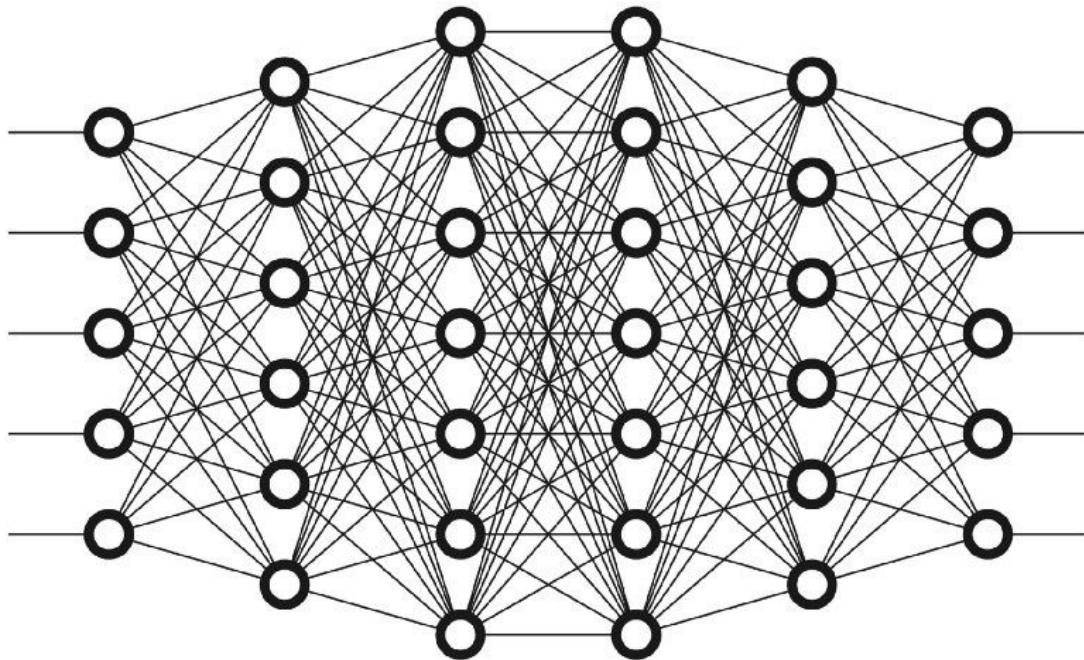
24 Khordad 1405

CE 417-Artificial Intelligence, Sharif Univ. of Tech.

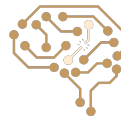
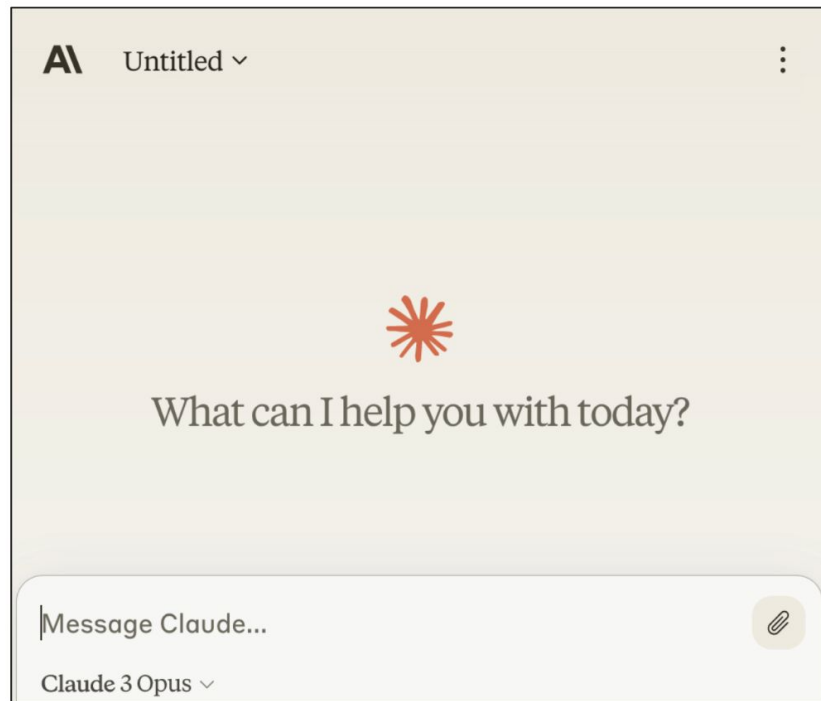
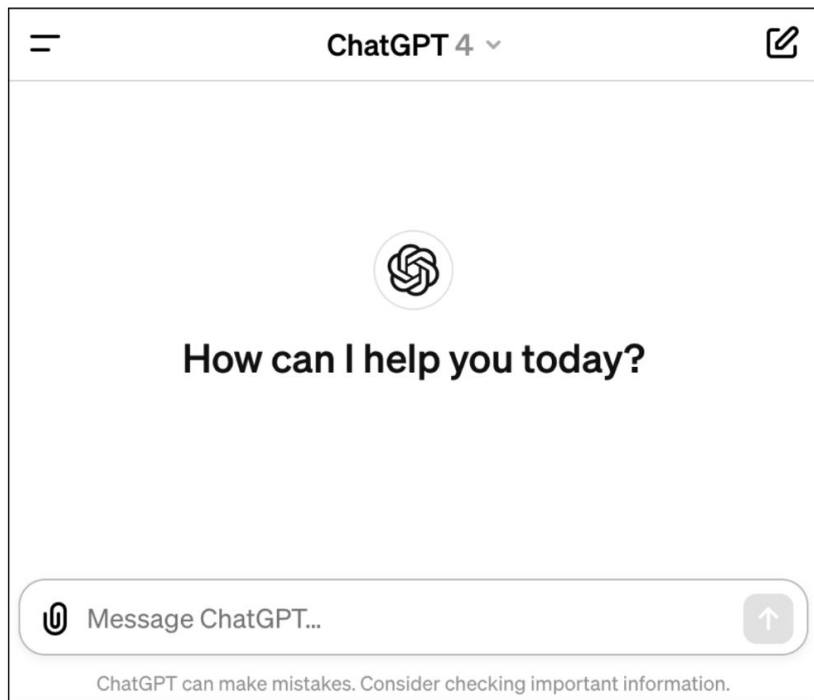
Most Slides have been adopted from Berkeley CS188 & SBU CI course Dr. Malek & SUT CE417 and CE717



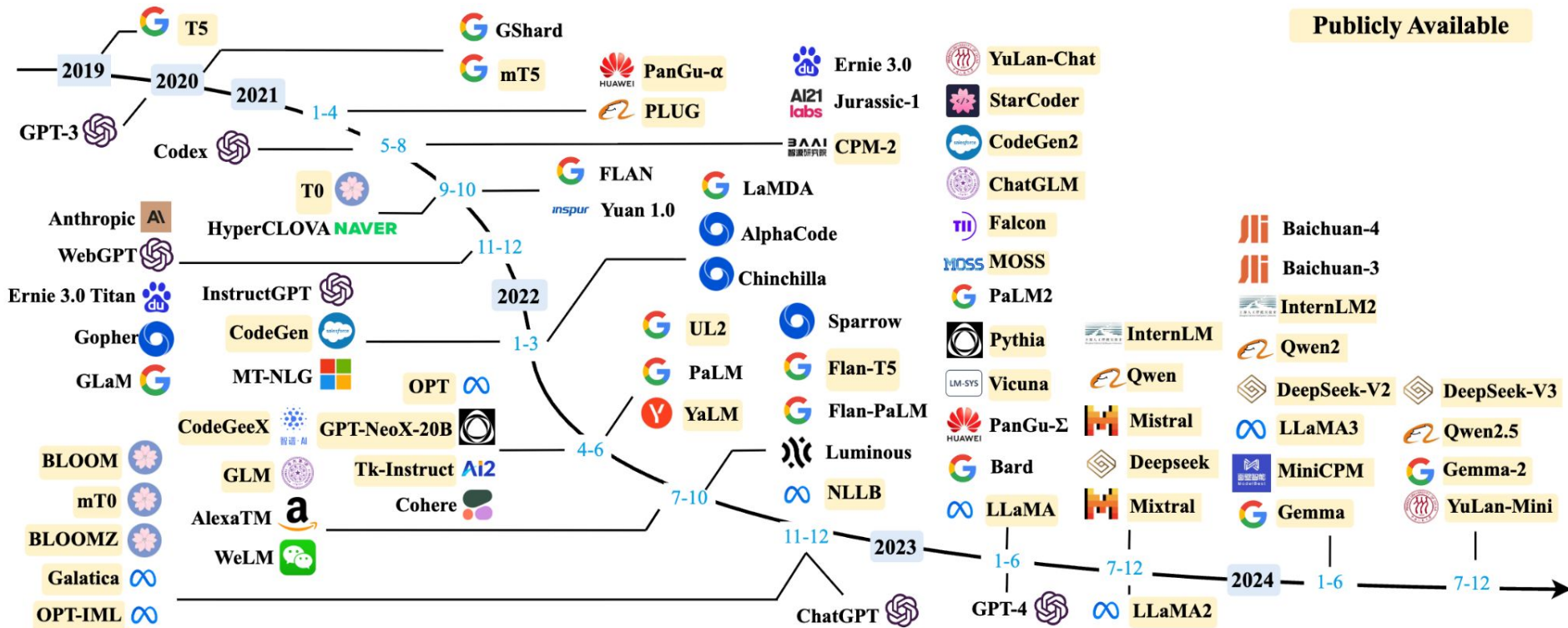
Deep Neural Network



Today's AI

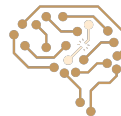


LLM Zoo



What is a Language Model?

- A probabilistic model that assigns a probability $P(w_1, w_2, \dots, w_n)$ to every finite sequences of tokens
- Generation from a language model: $x_{1:L} \sim p$



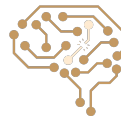
Autoregressive Language Models

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t | x_1, \dots, x_{t-1})$$

- $P(x_t | x_{1:t-1})$ is modeled efficiently (e.g., using a feedforward NN)
- Example:

$P(\text{He wants to know it})$

$= P(\text{He})P(\text{wants}|\text{He})P(\text{to}|\text{He wants})P(\text{know}|\text{He wants to})P(\text{it}|\text{He wants to know})$



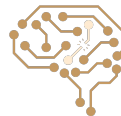
Autoregressive Language Models: Generation

- Generation: sample one token at a time given the tokens generated so far:

for $i=1,\dots,L$:

$$x_i \sim p_T(x_i|x_{1:i-1})$$

- $p_T(x_i|x_{1:i-1}) \propto p(x_i|x_{1:i-1})^{1/T}$ is an annealed conditional probability distribution
- $T \geq 0$ controls randomness



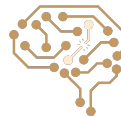
Autoregressive Language Models: Generation (cont.)

- **Conditional generation.** Specifying a prefix $x_{1:i}$, called a **prompt**, and sampling the rest $x_{i+1:L}$
- For example, generating with $T=0$ produces

Prompt: The, dog, eats

Completion ($T=0$): the bone

- Conditional generation unlocks the ability to solve a variety of tasks by simply changing the prompt.



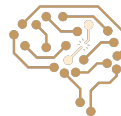
Neural Language Models

- Bengio et al. proposed neural language models in 2003:

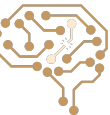
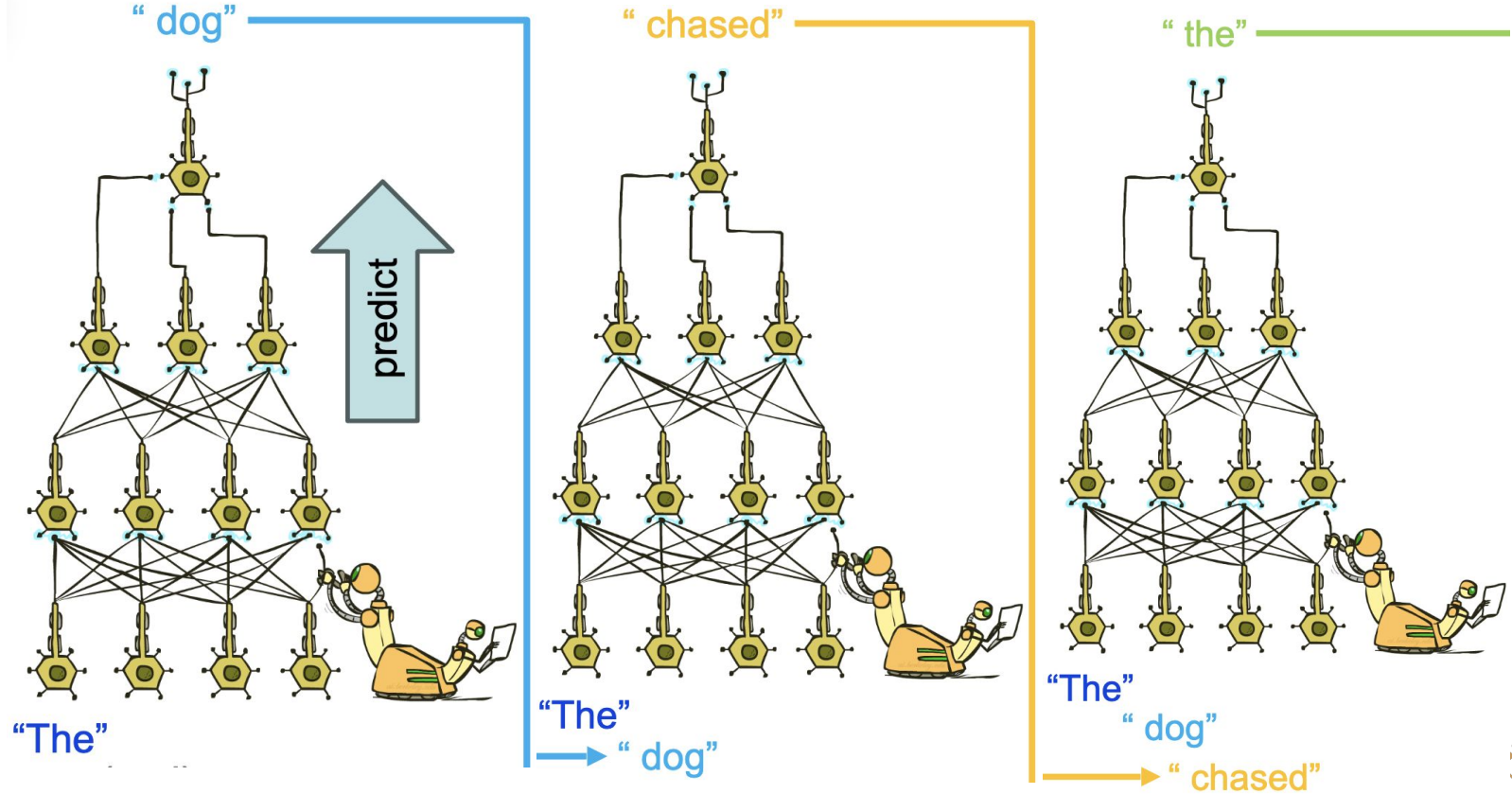
$$NN(w_1, \dots, w_n) \approx p(w_n | w_1, \dots, w_{n-1})$$

distributed feature vectors

- Neural language models are statistically efficient but computationally inefficient
 - Their training was not scalable

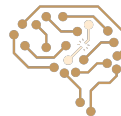


Autoregressive Language Model



Challenges of Using Deep NN for Language Modeling

- In natural language, we have words (tokens), but deep neural networks operates over vectors of real numbers
- Common deep neural networks consider fixed sized input, but sentences are not fixed size



Text Tokenization

GPT-3.5 & GPT-4

GPT-3 (Legacy)

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🍌🍌🍌🍌🍌

Sequences of characters commonly found next to each other may be grouped together: 1234567890

Text

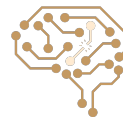
Token IDs

Tokens

57

Characters

252



Tokens to IDs

GPT-3.5 & GPT-4

GPT-3 (Legacy)

```
[8607, 4339, 2472, 311, 832, 4037, 11, 719, 1063, 1541, 956, 25, 3687,  
23936, 382, 35020, 5885, 1093, 100166, 1253, 387, 6859, 1139, 1690,  
11460, 8649, 279, 16940, 5943, 25, 11410, 97, 248, 9468, 237, 122, 271,  
1542, 45045, 315, 5885, 17037, 1766, 1828, 311, 1855, 1023, 1253, 387,  
41141, 3871, 25, 220, 4513, 10961, 16474, 15]
```

Text

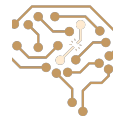
Token IDs

Tokens

57

Characters

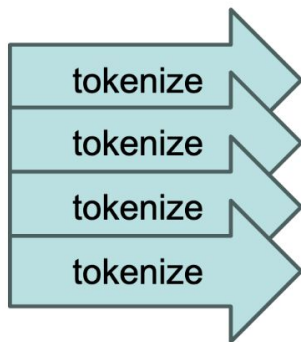
252



Token IDs to Embedding Vectors

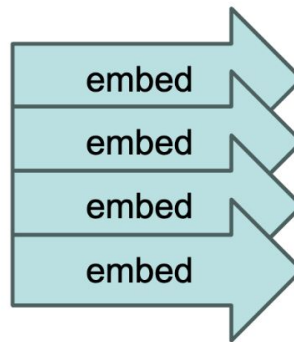
■ Input: some text

- "The"
- "dog"
- "chased"
- "the"



one-hot

[791]
[5679]
[62920]
[279]

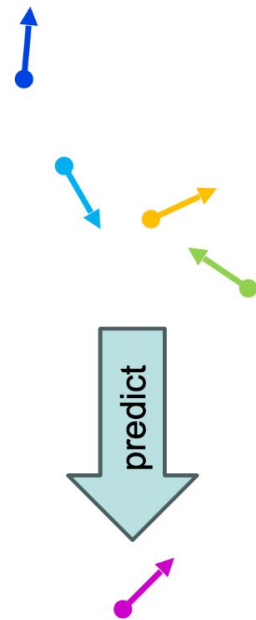
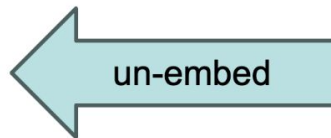


■ Output: more text

- "ball"



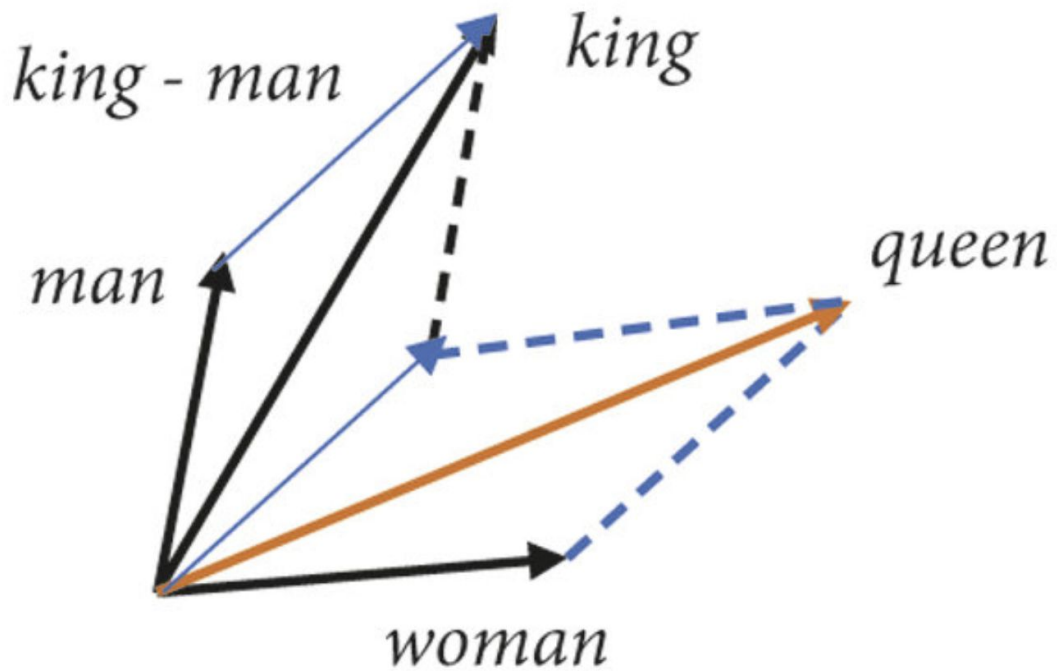
[5041]



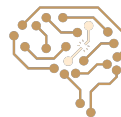
Word Embeddings



Word Embeddings

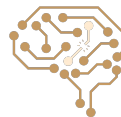
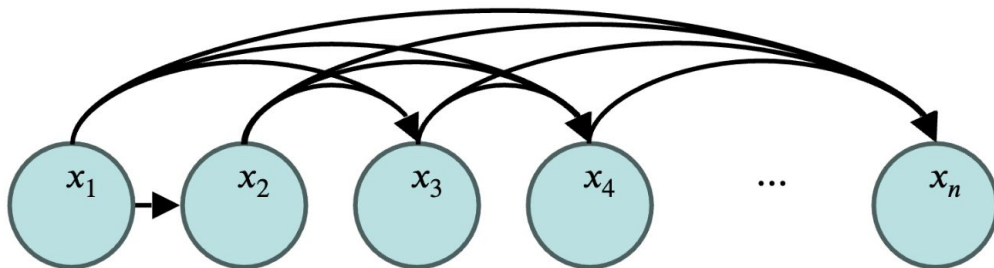


$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

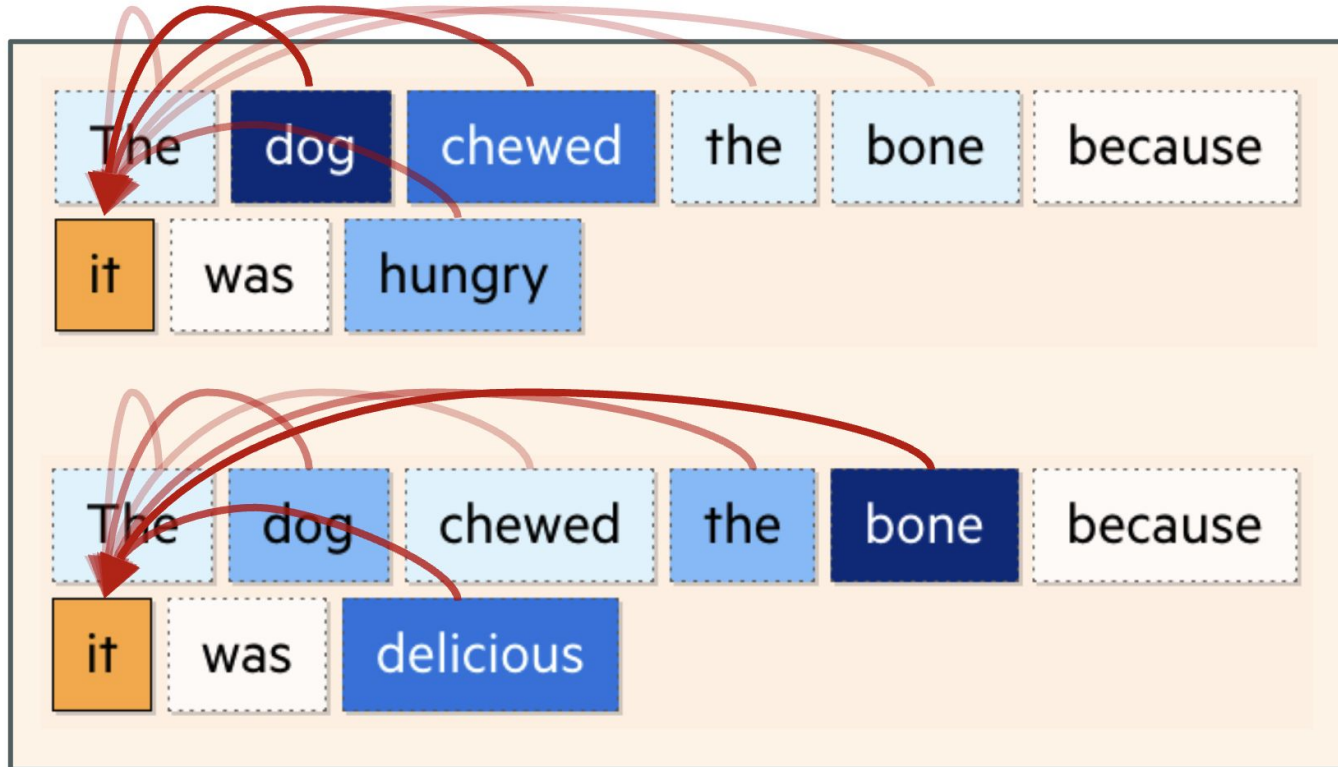


Recall: Autoregressive Language Model

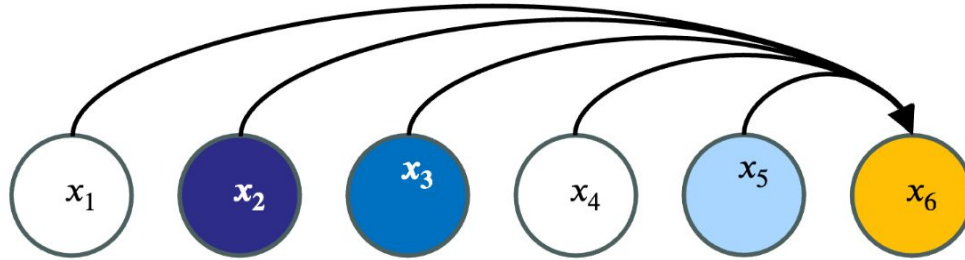
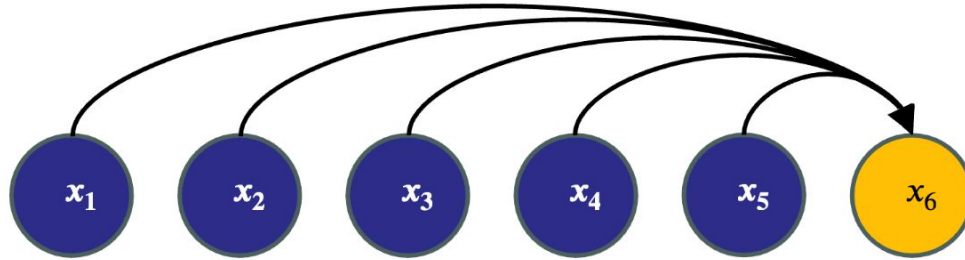
- Predict output one piece at a time (e.g. word, token, pixel, etc.)
- Concatenate: input + output
- Feed result back in as new input
- Repeat



Attention Idea



Attention Idea



- Instead of conditioning on *all* input tokens equally...
- Pay more attention to relevant tokens!



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

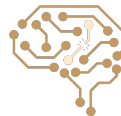
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

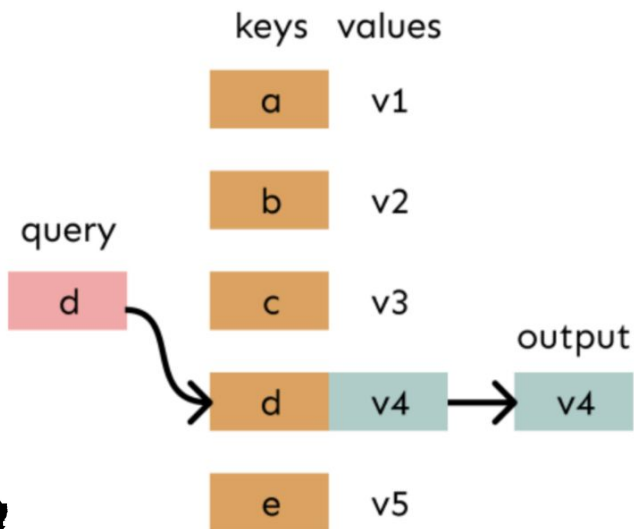
Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

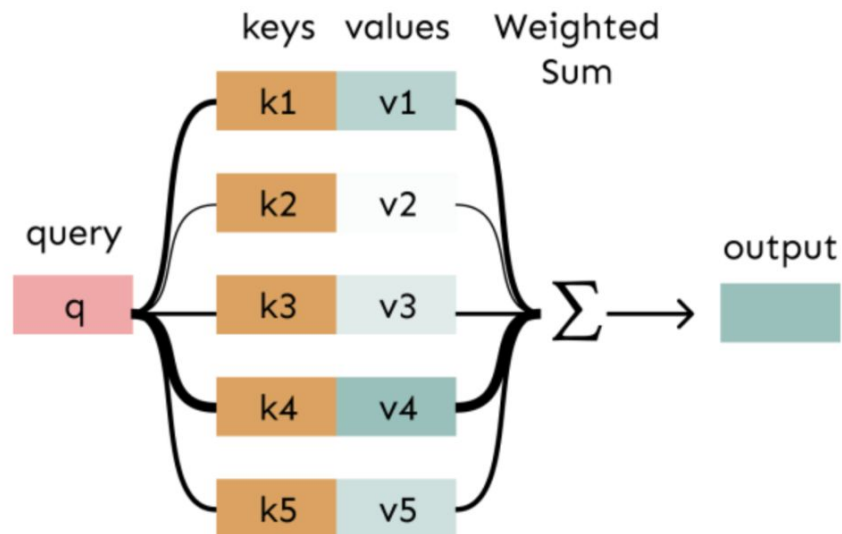


Attention as a Soft Lookup Table

In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



Attention Mechanism

Inputs:

Input vectors: x (shape: $N \times D$)

Operations:

Key vectors: $k = W_k^T x$

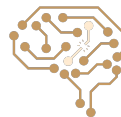
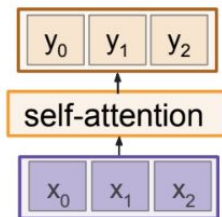
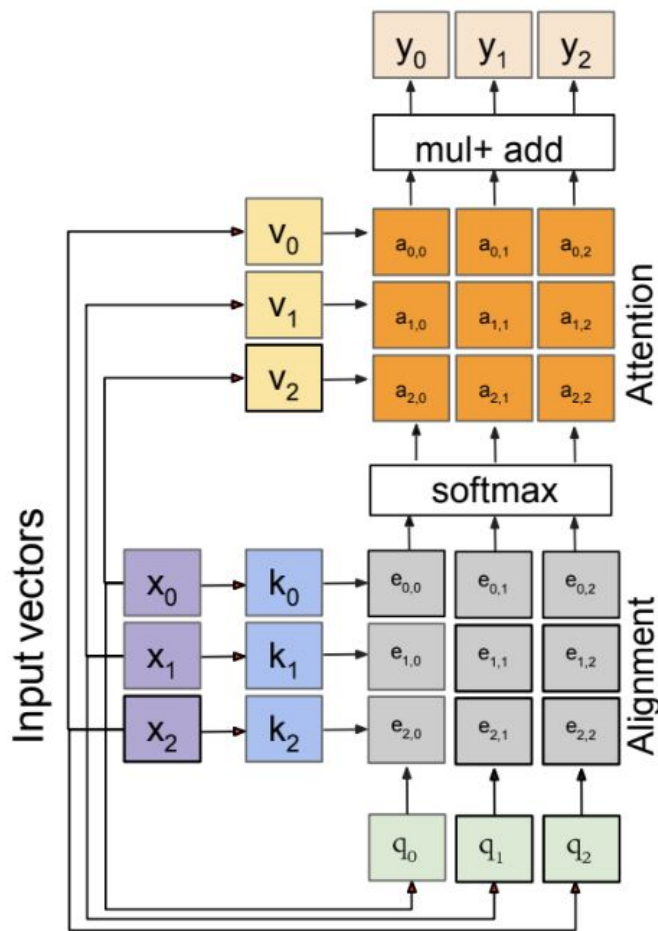
Value vectors: $v = W_v^T x$

Query vectors: $q = W_q^T x$

Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$

Attention: $a = \text{softmax}(e)$

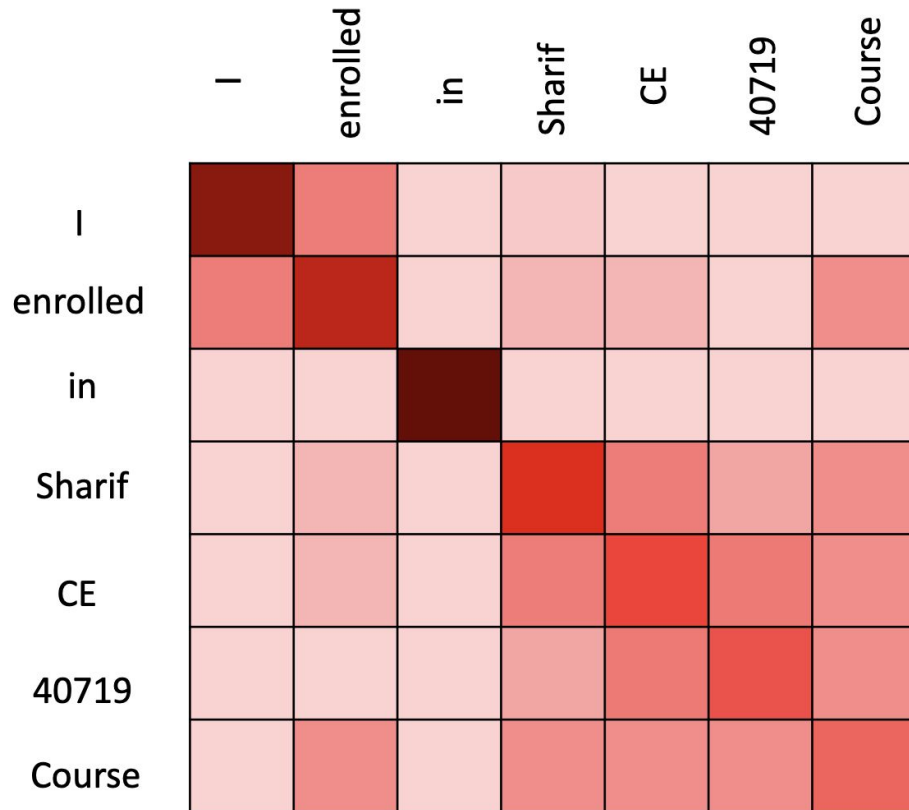
Output: $y_j = \sum_i a_{i,j} v_i$



Attention Map Example

Attention map: $\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right)$

Output: $\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right)V$



Attention Layer

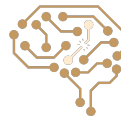
$$XW_k = K$$

$$XW_q = Q$$

$$XW_v = V$$

Attention map: $\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right)$

Output: $\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right)V$



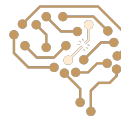
Example: Input Embeddings

- Suppose we have 3 tokens so far, want to predict the next (forth) token
- Here are token embeddings:

$$\text{Token 1: } x_1 = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix}$$

$$\text{Token 2: } x_2 = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix}$$

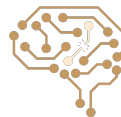
$$\text{Token 3: } x_3 = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix}$$



Example: Weight Matrices

- Query, Key, and Value weight matrices at the current layer:

$$W_Q = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix}$$

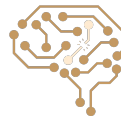


Example: Computing Query, Key, and Value For Token 1

$$q_1 = x_1 W_Q = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix}$$

$$k_1 = x_1 W_K = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix}$$

$$v_1 = x_1 W_V = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & 1.0 \end{bmatrix}$$

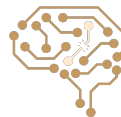


Example: Computing Query, Key, and Value For Token 2

$$q_2 = x_2 W_Q = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix}$$

$$k_2 = x_2 W_K = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix}$$

$$v_2 = x_2 W_V = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.5 \end{bmatrix}$$

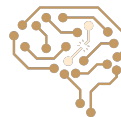


Example: Computing Query, Key, and Value For Token 3

$$q_3 = x_3 W_Q = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 2.0 & 1.0 \end{bmatrix}$$

$$k_3 = x_3 W_K = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 2.0 \end{bmatrix}$$

$$v_3 = x_3 W_V = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \end{bmatrix}$$

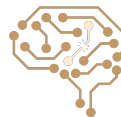


Example: Attention Scores for Token 1

The attention score s_{ij} between token i (using its query q_i) and token j (using its key k_j) is computed as the dot product $q_i \cdot k_j$. It is then scaled by dividing by $\sqrt{d_k}$ (which is **1.414**).

Scores for Token 1 (Query 1):

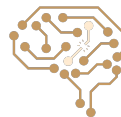
- **vs Token 1:** $s_{11} = (1.0)(0.0) + (0.0)(1.0) = 0.0 \xrightarrow{\text{scale}} \frac{0.0}{1.414} = 0.000$
- **vs Token 2:** $s_{12} = (1.0)(1.0) + (0.0)(1.0) = 1.0 \xrightarrow{\text{scale}} \frac{1.0}{1.414} = 0.707$
- **vs Token 3:** $s_{13} = (1.0)(1.0) + (0.0)(2.0) = 1.0 \xrightarrow{\text{scale}} \frac{1.0}{1.414} = 0.707$



Example: Attention Scores for Token 2

Scores for Token 2 (Query 2):

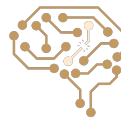
- vs Token 1: $s_{21} = (1.0)(0.0) + (1.0)(1.0) = 1.0$ $\xrightarrow{\text{scale}} \frac{1.0}{1.414} = 0.707$
- vs Token 2: $s_{22} = (1.0)(1.0) + (1.0)(1.0) = 2.0$ $\xrightarrow{\text{scale}} \frac{2.0}{1.414} = 1.414$
- vs Token 3: $s_{23} = (1.0)(1.0) + (1.0)(2.0) = 3.0$ $\xrightarrow{\text{scale}} \frac{3.0}{1.414} = 2.121$



Example: Attention Scores for Token 3

Scores for Token 3 (Query 3):

- vs Token 1: $s_{31} = (2.0)(0.0) + (1.0)(1.0) = 1.0$ $\xrightarrow{\text{scale}} \frac{1.0}{1.414} = 0.707$
- vs Token 2: $s_{32} = (2.0)(1.0) + (1.0)(1.0) = 3.0$ $\xrightarrow{\text{scale}} \frac{3.0}{1.414} = 2.121$
- vs Token 3: $s_{33} = (2.0)(1.0) + (1.0)(2.0) = 4.0$ $\xrightarrow{\text{scale}} \frac{4.0}{1.414} = 2.828$

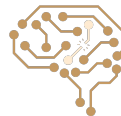


Example: Softmax Operation for Token 1

We apply the softmax equation $\alpha_{ij} = \frac{\exp(s'_{ij})}{\sum_k \exp(s'_{ik})}$ to convert the scaled scores into probabilities for each token.

Token 1's Attention Weights:

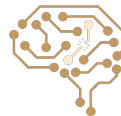
- Exponentials: $e^{0.000} = 1.000$, $e^{0.707} = 2.028$, $e^{0.707} = 2.028$
- Sum = **5.056**
- $\alpha_{11} = \frac{1.000}{5.056} = 0.198$, $\alpha_{12} = \frac{2.028}{5.056} = 0.401$, $\alpha_{13} = \frac{2.028}{5.056} = 0.401$



Example: Softmax Operation for Token 2

Token 2's Attention Weights:

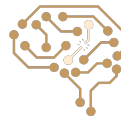
- Exponentials: $e^{0.707} = 2.028$, $e^{1.414} = 4.112$, $e^{2.121} = 8.339$
- Sum = **14.479**
- $\alpha_{21} = \frac{2.028}{14.479} = 0.140$, $\alpha_{22} = \frac{4.112}{14.479} = 0.284$, $\alpha_{23} = \frac{8.339}{14.479} = 0.576$



Example: Softmax Operation for Token 3

Token 3's Attention Weights:

- Exponentials: $e^{0.707} = 2.028$, $e^{2.121} = 8.339$, $e^{2.828} = 16.912$
- Sum = **27.279**
- $\alpha_{31} = \frac{2.028}{27.279} = 0.074$, $\alpha_{32} = \frac{8.339}{27.279} = 0.306$, $\alpha_{33} = \frac{16.912}{27.279} = 0.620$



Example: Output for Token 1

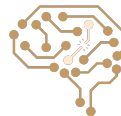
The final representation for each token z_i is computed by taking a weighted sum of all value vectors (v_j), using the softmax weights (α_{ij}) calculated above.

$$\text{Equation: } z_i = \alpha_{i1}v_1 + \alpha_{i2}v_2 + \alpha_{i3}v_3$$

For Token 1 (z_1):

$$z_1 = 0.198 \begin{bmatrix} 0.5 & 1.0 \end{bmatrix} + 0.401 \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} + 0.401 \begin{bmatrix} 1.5 & 1.5 \end{bmatrix}$$

$$z_1 = \begin{bmatrix} 0.099 & 0.198 \end{bmatrix} + \begin{bmatrix} 0.401 & 0.201 \end{bmatrix} + \begin{bmatrix} 0.601 & 0.601 \end{bmatrix} = \begin{bmatrix} 1.101 & 1.000 \end{bmatrix}$$

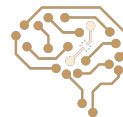


Example: Output for Token 2

For Token 2 (z_2):

$$z_2 = 0.140 \begin{bmatrix} 0.5 & 1.0 \end{bmatrix} + 0.284 \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} + 0.576 \begin{bmatrix} 1.5 & 1.5 \end{bmatrix}$$

$$z_2 = \begin{bmatrix} 0.070 & 0.140 \end{bmatrix} + \begin{bmatrix} 0.284 & 0.142 \end{bmatrix} + \begin{bmatrix} 0.864 & 0.864 \end{bmatrix} = \begin{bmatrix} 1.218 & 1.146 \end{bmatrix}$$

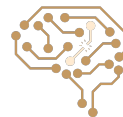


Example: Output for Token 3

For Token 3 (z_3):

$$z_3 = 0.074 \begin{bmatrix} 0.5 & 1.0 \end{bmatrix} + 0.306 \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} + 0.620 \begin{bmatrix} 1.5 & 1.5 \end{bmatrix}$$

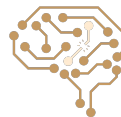
$$z_3 = \begin{bmatrix} 0.037 & 0.074 \end{bmatrix} + \begin{bmatrix} 0.306 & 0.153 \end{bmatrix} + \begin{bmatrix} 0.930 & 0.930 \end{bmatrix} = \begin{bmatrix} 1.273 & 1.157 \end{bmatrix}$$



Example: Matrix Formulation

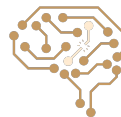
$$X = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$$

$$W_Q = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix}$$



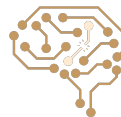
Example: Matrix Formulation (cont.)

$$Q = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \\ 2.0 & 1.0 \end{bmatrix}$$



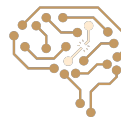
Example: Matrix Formulation (cont.)

$$K = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 1.0 \\ 1.0 & 2.0 \end{bmatrix}$$



Example: Matrix Formulation (cont.)

$$V = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & 1.0 \\ 1.0 & 0.5 \\ 1.5 & 1.5 \end{bmatrix}$$

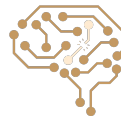


Example: Matrix Formulation (cont.)

$$S = QK^T$$

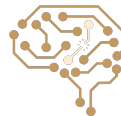
$$K^T = \begin{bmatrix} 0.0 & 1.0 & 1.0 \\ 1.0 & 1.0 & 2.0 \end{bmatrix}$$

$$S = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \\ 2.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.0 & 1.0 & 1.0 \\ 1.0 & 1.0 & 2.0 \end{bmatrix} = \begin{bmatrix} 0.0 & 1.0 & 1.0 \\ 1.0 & 2.0 & 3.0 \\ 1.0 & 3.0 & 4.0 \end{bmatrix}$$



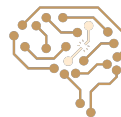
Example: Matrix Formulation (cont.)

$$S_{scaled} = \frac{QK^T}{\sqrt{d_k}} = \frac{1}{1.414} \begin{bmatrix} 0.0 & 1.0 & 1.0 \\ 1.0 & 2.0 & 3.0 \\ 1.0 & 3.0 & 4.0 \end{bmatrix} \approx \begin{bmatrix} 0.000 & 0.707 & 0.707 \\ 0.707 & 1.414 & 2.121 \\ 0.707 & 2.121 & 2.828 \end{bmatrix}$$



Example: Matrix Formulation (cont.)

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) = \begin{bmatrix} 0.198 & 0.401 & 0.401 \\ 0.140 & 0.284 & 0.576 \\ 0.074 & 0.306 & 0.620 \end{bmatrix}$$



Example: Predicting the Next Token

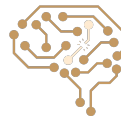
Final State of Last Token (z_3):

$$z_3 = [1.273 \quad 1.157]$$

We need a projection matrix (often called the vocabulary embedding matrix or Unembedding matrix, W_{vocab}) to map our 1×2 vector to a 1×5 vector, where 5 is our vocabulary size.

Let's define a sample 2×5 weight matrix for W_{vocab} :

$$W_{vocab} = \begin{bmatrix} 1.0 & -1.0 & 0.5 & 0.0 & 2.0 \\ 0.0 & 1.0 & -0.5 & 2.0 & -1.0 \end{bmatrix}$$



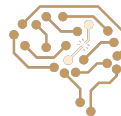
Example: Predicting the Next Token (cont.)

We multiply the final token's embedding by the W_{vocab} matrix to compute the unnormalized scores (logits) for every word in the vocabulary.

$$L = z_3 W_{vocab}$$

$$L = \begin{bmatrix} 1.273 & 1.157 \end{bmatrix} \begin{bmatrix} 1.0 & -1.0 & 0.5 & 0.0 & 2.0 \\ 0.0 & 1.0 & -0.5 & 2.0 & -1.0 \end{bmatrix}$$

$$L = \begin{bmatrix} 1.273 & -0.116 & 0.058 & 2.314 & 1.389 \end{bmatrix}$$

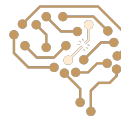


Example: Predicting the Next Token (cont.)

To convert these raw logits into a probability distribution over the vocabulary, we apply the Softmax function.

$$P_i = \frac{\exp(L_i)}{\sum_{j=1}^V \exp(L_j)}$$

$$P = [0.182 \quad 0.045 \quad 0.054 \quad 0.515 \quad 0.204]$$



Example: Predicting the Next Token (cont.)

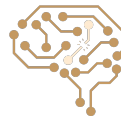
To generate the next word, the model selects the index of the vocabulary with the highest probability (greedy decoding), or samples from this distribution.

$$\text{Next Token ID} = \text{argmax}(P)$$

Looking at the probability array P :

- Token 1: **18.2%**
- Token 2: **4.5%**
- Token 3: **5.4%**
- Token 4: **51.5%**
- Token 5: **20.4%**

$$\text{argmax} \left([0.182 \quad 0.045 \quad 0.054 \quad 0.515 \quad 0.204] \right) = \text{Index 4}$$



Causal (Masked) Attention Layer

- Masking the future

Operations:

Key vectors: $\mathbf{k} = W_k^T \mathbf{x}$

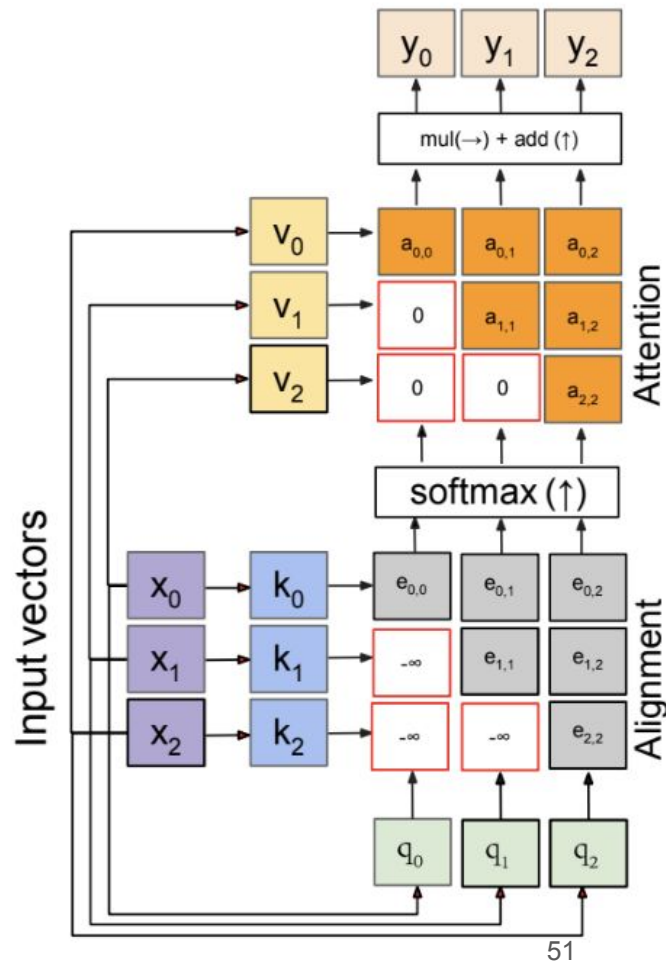
Value vectors: $\mathbf{v} = W_v^T \mathbf{x}$

Query vectors: $\mathbf{q} = W_q^T \mathbf{x}$

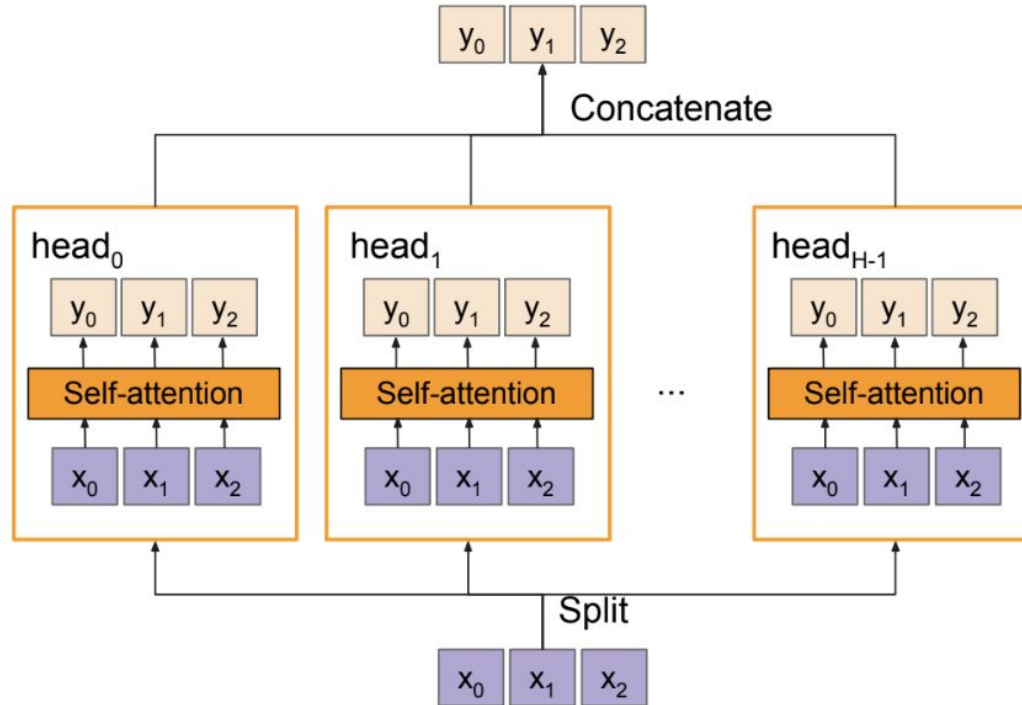
Alignment: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

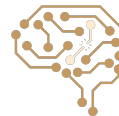
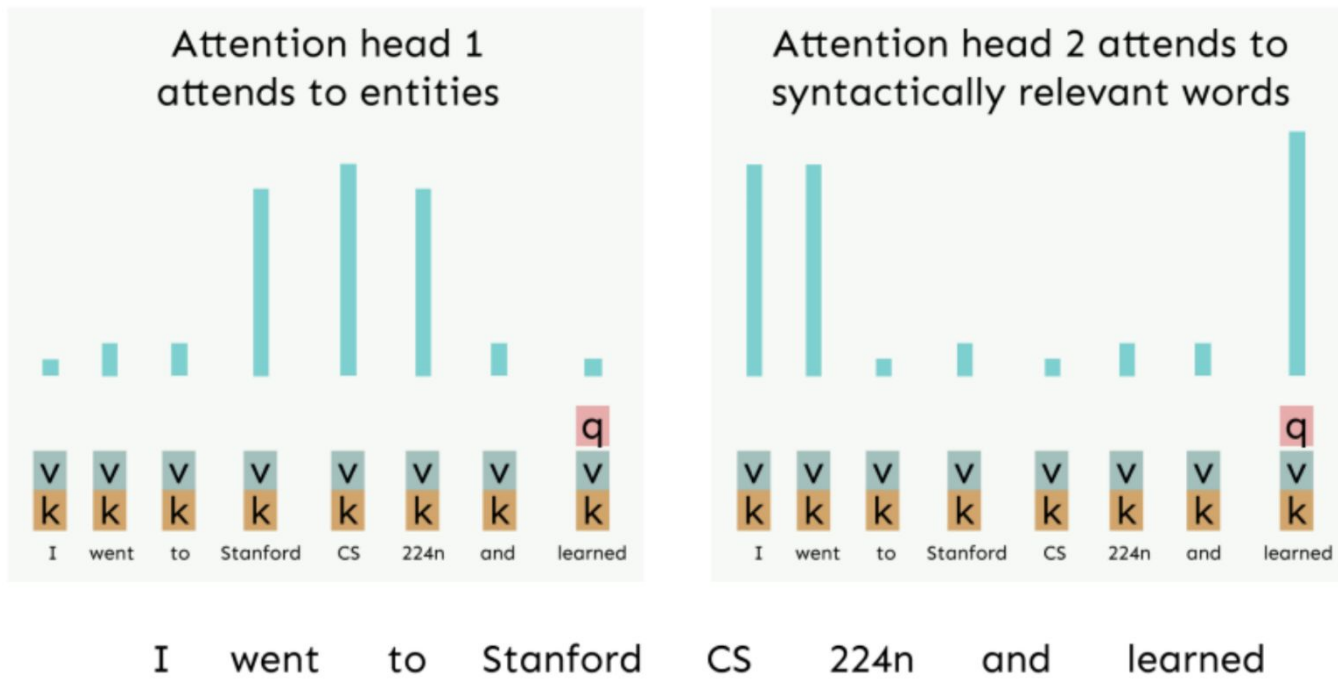
Output: $y_j = \sum_i a_{i,j} \mathbf{v}_i$



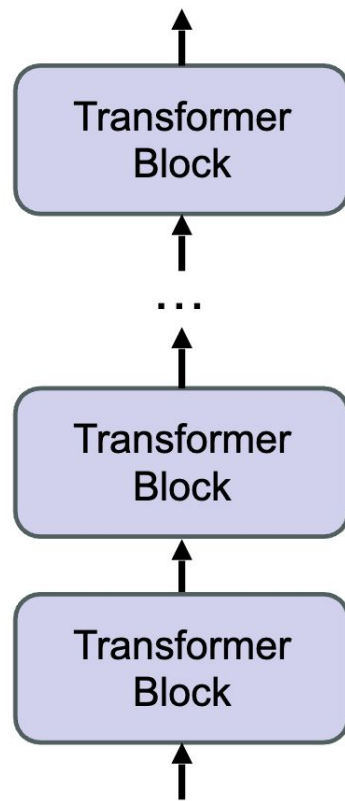
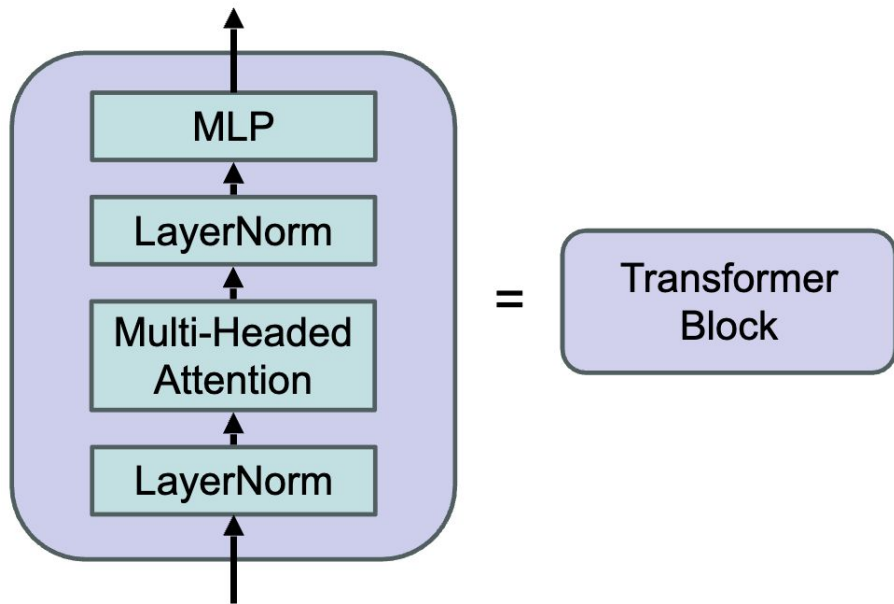
Multi-head Attention Layer



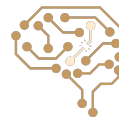
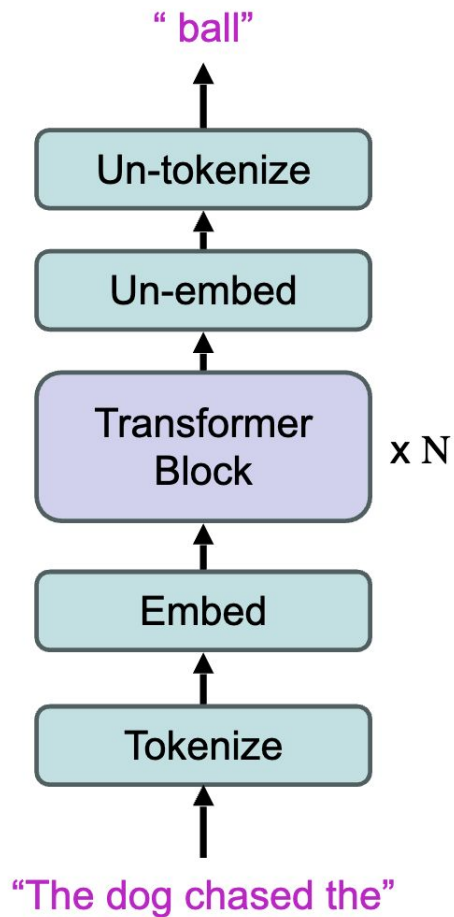
Multi-head Attention Layer (cont.)



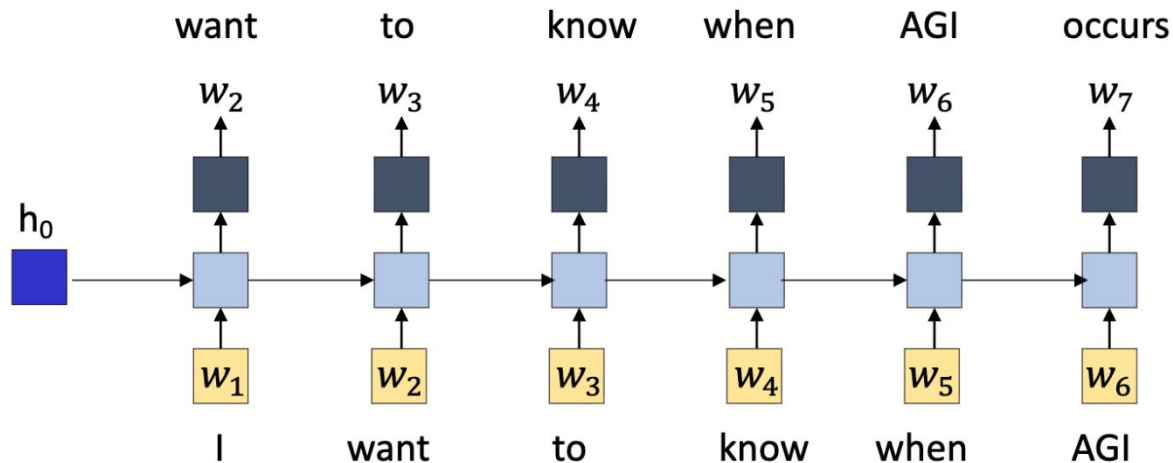
Transformer



Transformer (cont.)



How to Train?



- Learn a model that can predict the next token given a sequence of tokens
- Maximize the log-likelihood of the training data



How to Train?

- $\hat{y} \in \mathbb{R}^{|V|}$ is a probability distribution over the vocabulary
- Cross entropy loss function at location t of the sequence:

$$E_t = - \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}$$

$y_{t,j} = 1$ when w_t must be the word j of vocabulary

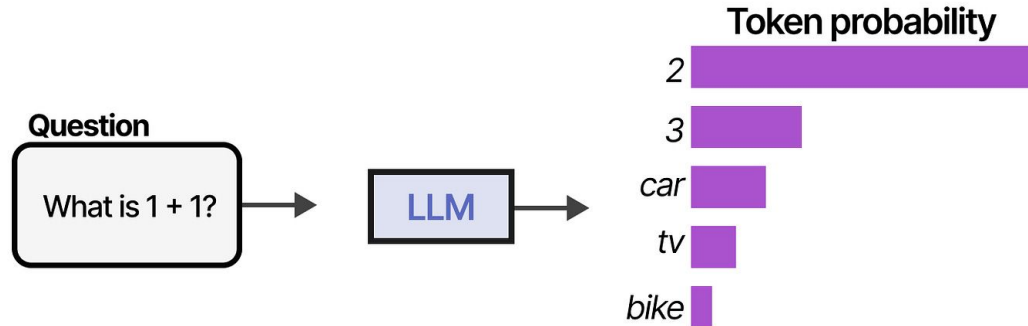
- Cost function over the entire sequence:

$$E = - \frac{1}{T} \sum_{t=1}^T \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}$$

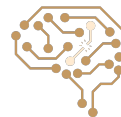
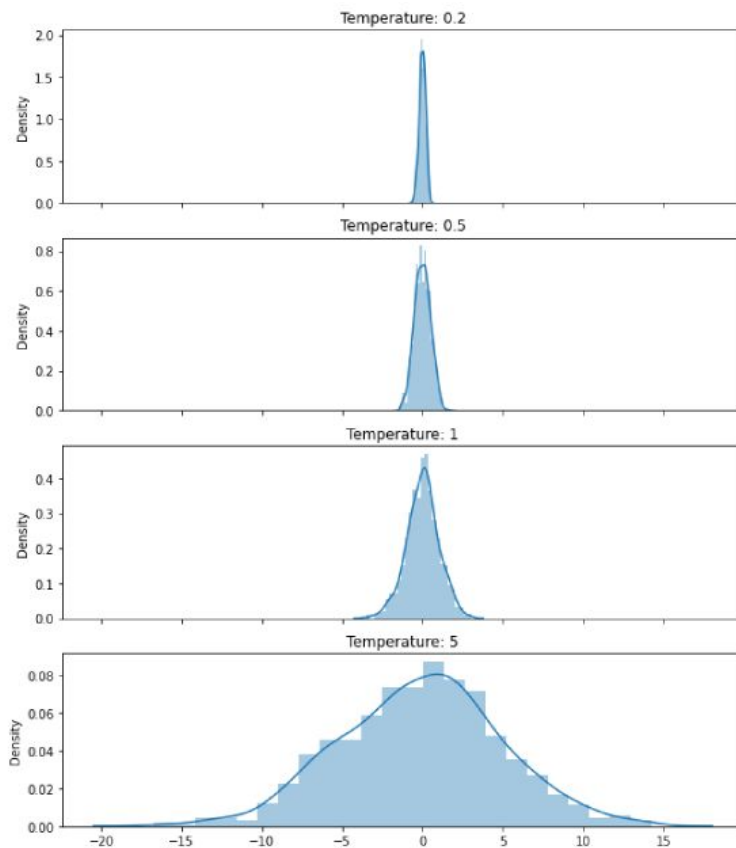


Decoding in LLMs

- LLM performs next token prediction
- At each time, for predicting the new/next token, LLM produce a probability distribution over all tokens of dictionary
- Then, following a sampling strategy, the next token is sampled from the provided distribution

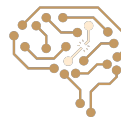
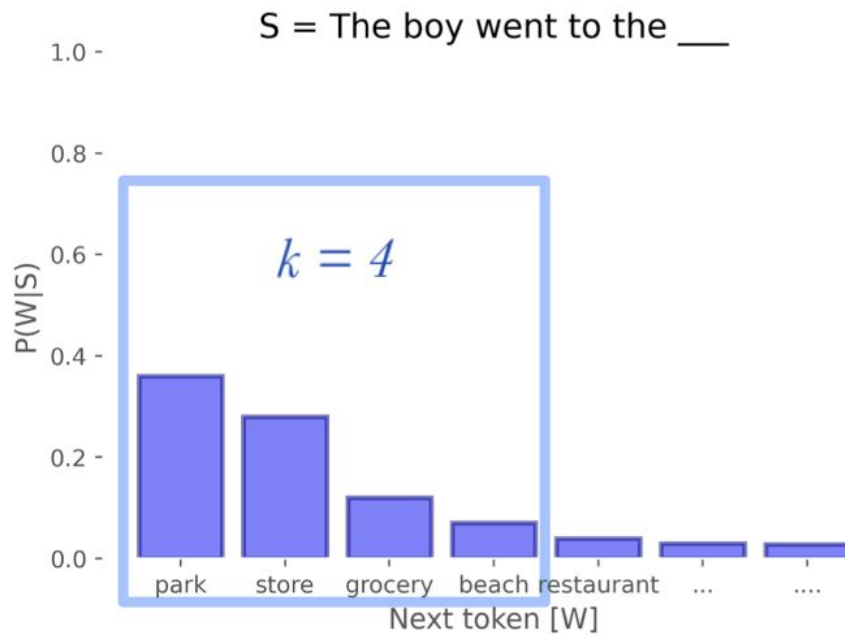


Decoding Temperature

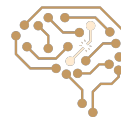
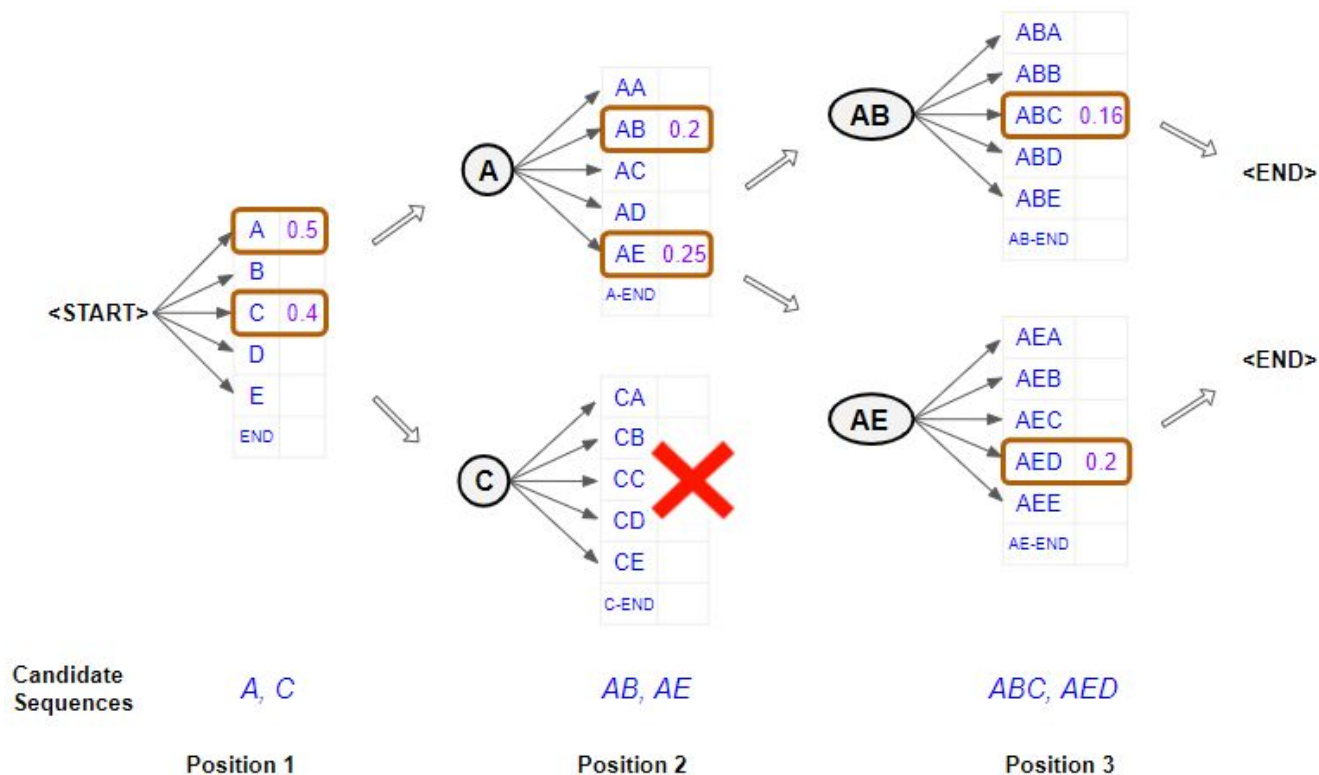


Decoding Strategy

- Greedy
- Top-k
- Top-p



Beam Search



Next Session: Convolutional Neural Networks (CNNs)

